

RABBIT Bipedal Robot

A MATLAB Simulation and Hybrid Zero Dynamics Gait Framework

Technical Documentation

Mohammad Malek

August 2, 2026

Abstract

This document describes a modular MATLAB framework for modeling, simulation, control, and trajectory optimization of the RABBIT planar five-link underactuated biped. The centerpiece is a working Hybrid Zero Dynamics (HZD) gait-optimization pipeline that produces dynamically feasible, periodic walking gaits and animates them. We cover the robot model, the continuous and hybrid dynamics, the HZD control formulation, the nonlinear program solved with `fmincon` in both of its transcriptions — single shooting and Hermite–Simpson direct collocation — the software architecture and data flow, a per-folder file reference, and reproducibility notes. Deviations between the documented and the as-built code (broken legacy entry points, orphaned files) are flagged explicitly.

1 Overview

RABBIT is a planar, five-link, *underactuated* biped: a torso plus two legs, each with a hip and a knee. Walking is modelled as a hybrid dynamical system — continuous single-support (swing) dynamics punctuated by a discrete impact when the swing foot strikes the ground. A gait is designed by imposing *virtual constraints* (B-spline reference trajectories for the actuated joints) and choosing their coefficients, together with the periodic initial state, so that the closed-loop system admits a stable periodic orbit while minimizing actuator effort.

Dependencies. MATLAB with the **Symbolic Math Toolbox** (only to *regenerate* the dynamics; the generated files are committed) and the **Optimization Toolbox** (`fmincon`, SQP — required for the gait pipeline).

Working entry point.

<code>startup</code>	<code>% add all folders to the path</code>
<code>cd Trajectory_Optimization</code>	
<code>rabbit_hzd_trajectory_optimization</code>	<code>% optimize one gait, inspect, animate</code>

A gait library across walking speeds is built with `build_gait_library.m`. See Section 15 for legacy entry points that no longer run.

2 Robot model

The generalized coordinates (7 DOF) are

$$q = [p_x \quad y \quad q_t \quad q_1 \quad q_2 \quad q_3 \quad q_4]^\top, \quad x = \begin{bmatrix} q \\ \dot{q} \end{bmatrix} \in \mathbb{R}^{14}. \quad (1)$$

symbol	meaning	actuated
p_x, y	torso base position (world frame)	no
q_t	torso pitch angle	no
q_1, q_2	stance-leg hip, knee	yes
q_3, q_4	swing-leg hip, knee	yes

Table 1: Generalized coordinates. Four actuators drive $q_1 \dots q_4$; the floating base (p_x, y, q_t) is unactuated, giving one degree of underactuation in single support.

The input map (`Dynamics/input_matrix.m`) is the constant

$$B = \begin{bmatrix} 0_{3 \times 4} \\ I_4 \end{bmatrix} \in \mathbb{R}^{7 \times 4}. \quad (2)$$

Physical parameters. There is no separate model/parameter file; the physical parameters are baked into the symbolic derivation (`Dynamics/rabbit_energy_model_generalized_Lagrange.m`, nested `Mass_Properties`): torso 10 kg / 0.75 m; each leg link (thigh, shank) 5 kg / 0.5 m; gravity $g = 9.8062 \text{ m/s}^2$.

Sign convention (important). The world-frame Z axis is *up-positive* (ground = 0, above ground > 0), but y is stored *down-positive* in the generalized coordinates, so the world-frame hip height is $-y$. All kinematic helpers (`P_st`, `P_sw`, `Tt`, `get_body_points`) return world-frame, up-positive heights. Mixing the two conventions has historically caused sign bugs.

The name y follows the symbolic derivation, where the second generalized coordinate is declared as `y` (`rabbit_energy_model_generalized_Lagrange.m`, line 9), and the base transform carries it as $-y$ in the world-vertical row. **Note the symbol clash:** y also denotes the *virtual-constraint output* of Section 5. They are told apart by context and by arity — the output is always written with an argument, $y(q)$ or $y(x)$, and is a 4-vector; the coordinate is a scalar and appears only inside q .

3 Continuous dynamics

The unconstrained equations of motion are

$$M(q) \ddot{q} + V(q, \dot{q}) + G(q) = B u, \quad (3)$$

with $M(q)$ the mass/inertia matrix (`Dynamics/M.m`), $V(q, \dot{q})$ the Coriolis/centrifugal vector (`Dynamics/V.m`, called as `V([q;dq])`), and $G(q)$ the gravity vector (`Dynamics/G.m`).

Single-support (constrained) dynamics. During swing the stance foot is pinned by the holonomic contact constraint $p_{\text{st}}(q) = \text{const}$, i.e. at the acceleration level

$$J_{\text{st}}(q) \ddot{q} + \dot{J}_{\text{st}} \dot{q} = 0, \quad J_{\text{st}} = \frac{\partial p_{\text{st}}}{\partial q} \in \mathbb{R}^{2 \times 7}. \quad (4)$$

`Dynamics/rabbit_constrained_dynamics.m` solves the resulting saddle-point (KKT) system for the accelerations and the contact wrench λ :

$$\begin{bmatrix} M & -J_{\text{st}}^\top \\ J_{\text{st}} & 0 \end{bmatrix} \begin{bmatrix} \ddot{q} \\ \lambda \end{bmatrix} = \begin{bmatrix} B u - V - G \\ -\dot{J}_{\text{st}} \dot{q} \end{bmatrix}. \quad (5)$$

4 Hybrid model

Position is continuous across impact; only velocity jumps.

- **Guard / event** (`Contact/rabbit_impact_event.m`): the swing-foot height $p_{\text{sw},z}(q)$ crosses zero from above.
- **Impact map** (`Contact/rabbit_impact_map.m`): momentum-conserving reset enforcing zero post-impact swing-foot velocity,

$$\begin{bmatrix} M & -J_{\text{sw}}^\top \\ J_{\text{sw}} & 0 \end{bmatrix} \begin{bmatrix} \dot{q}^+ \\ \lambda \end{bmatrix} = \begin{bmatrix} M \dot{q}^- \\ 0 \end{bmatrix}. \quad (6)$$

- **Reset map** (`Reset_Map/rabbit_reset_map.m`): relabels stance/swing legs (`relabel_state.m`) and re-plants the new stance foot on the ground ($y \leftarrow p_{\text{st},z}$). The horizontal coordinate is intentionally not shifted, so the robot walks forward continuously in the world frame.

5 Phase variable and Hybrid Zero Dynamics

Phase variable (linear in q). The gait “clock” is the absolute stance-leg angle, *linear* in q :

$$\theta(q) = q_t + q_1 + \frac{1}{2}q_2 = c q, \quad c = \begin{bmatrix} 0 & 0 & 1 & 1 & \frac{1}{2} & 0 & 0 \end{bmatrix}. \quad (7)$$

This is implemented in `Controller/theta_of_q.m`; its gradient $\partial\theta/\partial q = c$ is the constant returned by `Controller/dtheta_dq_of.m`. Because both leg links have length $\frac{1}{2}$, the geometric hip-to-foot angle `atan2` equals (7) exactly for all physical poses ($|q_2| < \pi$); the linear form is preferred because *it cannot wrap at $\pm\pi$* and matches Westervelt’s Hypothesis HH6, on which the HZD decoupling analysis relies.

Virtual constraints. The normalized phase is

$$s = \frac{\theta - \theta^-}{\theta^+ - \theta^-} \in [0, 1], \quad (8)$$

where θ^- is the phase at step start and θ^+ (`p.theta_plus`) at touchdown. The four actuated joints track a clamped cubic B-spline reference $y_d(s)$,

$$y(q) = q_{4:7} - y_d(s), \quad u = -K_p y - K_d \dot{y}, \quad (9)$$

with $K_p = 400$, $K_d = 40$. The B-spline is evaluated by `bspline_eval.m` (basis functions from `BSpline.m` / `BSpline_derivative.m`); the whole closed loop is assembled in `Dynamics/hzd_closed_loop_ode.m` whose augmented state $\xi = [x; \text{torque cost}]$ integrates the running cost alongside the dynamics.

6 The Chapter 3 package: three core functions

`Chapter3/` is a separate, from-scratch implementation of the Chapter 3 pipeline. It reuses only the symbolically generated model in `Dynamics/`; everything above that layer is new. Three of its functions contain almost all of the non-obvious reasoning in the package, and are documented here in full. A per-function inventory and call graph is in `docs/CH3_FUNCTION_MAP.md`.

6.1 ch3_control_affine: building f and g

Chapter 3 requires the continuous phase in control-affine form, $\dot{x} = f(x) + g(x)u$. This is stronger than being able to evaluate $\ddot{q}$ for a given u : three later stages need the u -dependence separated *symbolically* — the decoupling matrix $L_g L_f y = J_y \ddot{q}_{\text{in}}$ of Section 6, the requirement that $\ddot{y}$ be affine in u so the stage-7 problem is a QP at all, and the requirement that the ground reaction force be affine in u so the friction cone is a *linear* inequality.

Starting from (3) with the stance foot pinned, the contact wrench enters as the generalized force $J_{\text{st}}^\top \lambda$, giving the saddle-point system

$$\underbrace{\begin{bmatrix} M & -J^\top \\ J & 0 \end{bmatrix}}_A \begin{bmatrix} \ddot{q} \\ \lambda \end{bmatrix} = \begin{bmatrix} Bu - V - G \\ -\dot{J}\dot{q} \end{bmatrix}, \quad J \equiv J_{\text{st}}. \quad (10)$$

The contact force is therefore not modelled but *solved for*, as the Lagrange multiplier of the same system that yields the accelerations.

The five right-hand sides. A contains no u , and u enters the right-hand side linearly. One factorization with five right-hand sides — one drift column and four input directions — therefore recovers the whole affine structure by superposition:

```
rhs = [ [-V_vec - G_vec], B_mat ; ...
        [-Jdotdq],      zeros(nc, nu) ];
sol = A \ rhs;          % (nq+nc) x (1+nu)
```

$$\ddot{q} = \ddot{q}_{\text{drift}} + \ddot{q}_{\text{in}} u, \quad \lambda = \lambda_{\text{drift}} + \lambda_{\text{in}} u, \quad (11)$$

$$f(x) = \begin{bmatrix} \dot{q} \\ \ddot{q}_{\text{drift}} \end{bmatrix}, \quad g(x) = \begin{bmatrix} 0_{7 \times 4} \\ \ddot{q}_{\text{in}} \end{bmatrix}. \quad (12)$$

This is exact — not linearized, not finite-differenced; it is the solution of (10), rearranged. The structural zero in the top block of g (torque cannot change position instantaneously) is precisely what makes the outputs relative degree 2.

Finite-differencing g instead would cost five solves *plus* truncation error, and every downstream quantity inherits it: $L_g L_f y \rightarrow u_{\text{ff}} \rightarrow$ the collocation cost $\rightarrow$ every gradient `fmincon` estimates. Measured, the batched solve is also $3.5\times$ faster ($9.6\mu\text{s}$ versus $33.2\mu\text{s}$).

Numerical character. Over all nodes of the reference gait, $\text{cond}(A) \in [457, 491]$; over 4000 random poses the median is 428 and the maximum 1.07×10^3 , with J_{st} never losing rank. Two structural notes:

- A is *not* symmetric ($\|A - A^\top\|_\infty = 5.88$), because the upper-right block is $-J^\top$ while the lower-left is $+J$. Negating one block row would give the symmetric indefinite form admitting $LDL^\top$. The present arrangement is equally valid and returns λ directly in the desired convention — the force the *ground applies to the robot*, $F_z > 0$ pushing up (measured $F_z = +39.75\text{ N}$ at $u = 0$). At 9×9 the solver choice is immaterial.
- **The solve is unguarded.** `ch3_io_lin` checks `rcond` and falls back to `pinv`; this function checks nothing. The justification holds — M is positive definite always and J_{st} for a planar point foot is generically full row rank, with zero rank failures in 4000 random poses — but a singular A would propagate NaN into an ODE step rather than raise a diagnosis.

Two details worth preserving. The local names `M_mat`, `V_vec`, `G_vec` avoid shadowing the functions `M`, `V`, `G` on the path — writing `M = M(q)` would make the next call in scope index the matrix instead. And `nc = size(J,1)` reads the contact count from the Jacobian rather than hardcoding it, so a different contact model propagates automatically. The `aux` struct is assembled only when `nargout > 2`, which matters when `ode45` calls the function tens of thousands of times per step.

6.2 ch3_io_lin: from a nonlinear robot to $\ddot{y} = \mu$

The outputs have relative degree 2: u is a torque, so it cannot appear until one has differentiated down to $\ddot{q}$. Because $\partial s / \partial q$ is a *constant* row (Hypothesis HH6, (7)), the derivative of the output Jacobian collapses to a single term,

$$\dot{J}_y = -\frac{d^2 y_d}{ds^2} \dot{s} \frac{\partial s}{\partial q}, \quad \dot{J}_y \dot{q} = -\frac{d^2 y_d}{ds^2} \dot{s}^2, \quad (13)$$

so that, substituting $\ddot{q} = \ddot{q}_{\text{drift}} + \ddot{q}_{\text{in}} u$,

$$\ddot{y} = \underbrace{J_y \ddot{q}_{\text{drift}} - \frac{d^2 y_d}{ds^2} \dot{s}^2}_{L_f^2 y \quad (4 \times 1)} + \underbrace{J_y \ddot{q}_{\text{in}}}_{L_g L_f y \quad (4 \times 4)} u. \quad (14)$$

Were θ nonlinear in q , $\dot{J}_y$ would additionally carry the *Hessian of the phase* — a 7×7 second-derivative tensor to build and differentiate at every timestep. That is the concrete value of the linear phase variable. Both terms are exact: $\ddot{q}_{\text{in}}$ from (10), $d^2 y_d / ds^2$ analytically from the Bézier basis.

Inverting the decoupling matrix gives

$$u = u_{\text{ff}} + (L_g L_f y)^{-1} \mu, \quad u_{\text{ff}} = -(L_g L_f y)^{-1} L_f^2 y \implies \ddot{y} = \mu. \quad (15)$$

Verified independently by central-differencing $\dot{y}(x)$ along the true flow $\dot{x} = f + gu$, never reusing the $\ddot{y}$ formula: $u = u_{\text{ff}}$ gives $\max |\ddot{y}| = 2.55 \times 10^{-9}$; a requested $\mu = [1.5, -2, 0.7, 3]^\top$ is delivered to 2.48×10^{-9} ; and $\mu = e_k$ moves only channel k , reproducing the identity to 10^{-9} . Peak $|u_{\text{ff}}|$ over the reference step is 106.6 Nm. Every controller in stages 5–8 differs *only* in how it selects μ .

What is not linearized. On the constraint manifold single support is 10-dimensional while $\eta = [y; \dot{y}]$ is 8, leaving a 2-dimensional zero dynamics $(\theta, \dot{\theta})$. This is *partial* feedback linearization, and partial by necessity: full-state feedback linearization requires full actuation, and there are four actuators for five effective DOF. Feedback handles the transverse directions; the choice of α handles the pendulum.

Conditioning, and a correction to the source comment. Measured: over the gait $\text{rcond}(L_g L_f y) \in [5.61, 8.34] \times 10^{-3}$ ($\text{cond} \in [80, 128]$); over 20 000 random states spanning the full $\pm\pi$ joint box the worst value is 8.05×10^{-6} . **The pinv fallback never fires** — zero times in 20 000 states. It is genuinely defensive.

The source comment attributes near-singularity to “*knee lock, or if dy_d/ds drives J_y toward a rank-deficient combination.*” Measured, the two halves do not fare equally:

- **Knee lock: not supported.** Driving the stance knee to full extension leaves conditioning flat ($q_2 = 1.347 \rightarrow \text{rcond } 5.76 \times 10^{-3}$; $q_2 = 0 \rightarrow 7.03 \times 10^{-3}$). A 121×121 grid over *both* knees across $(-\pi, \pi)$ puts the worst case at $\text{rcond} = 2.14 \times 10^{-7}$ at $(q_2, q_4) = (2.125, 1.866)$ — deep double-knee *flexion* ($\approx 122^\circ$ and $\approx 107^\circ$), not extension, and still five orders above the 10^{-12} threshold.

- **Profile slope: supported.** Stretching α about α_0 degrades conditioning monotonically: a $100\times$ steeper profile costs about 1.75 orders of magnitude of `rcond` ($5.76 \times 10^{-3} \rightarrow 1.01 \times 10^{-4}$).

The docstring has since been corrected accordingly: it now states the profile-slope mechanism, records that knee angle barely matters, and notes that the `pinv` fallback has never been observed to fire.

6.3 `ch3_impact`: the same matrix, a different meaning

The impact solves a system structurally identical to (10) — same saddle-point shape, same M , a foot Jacobian and a multiplier — while everything it *means* differs.

	<code>ch3_control_affine</code>	<code>ch3_impact</code>
unknowns	accelerations $\ddot{q}$	post-impact velocities $\dot{q}^+$
multiplier	contact <i>force</i> [N]	contact <i>impulse</i> [Ns]
Jacobian	J_{st} , pinned foot	J_{sw} , foot about to land
right side	forces $Bu - V - G$	momentum $M\dot{q}^-$
balance	force	momentum

Table 2: The two KKT systems of the hybrid model. Configuration q does not appear on the right of the impact system because it is *continuous* through impact; M and J_{sw} are both evaluated at the pre-impact configuration. The bottom row imposes $J_{\text{sw}}\dot{q}^+ = 0$: the foot strikes and sticks, with no rebound and no slip.

Measured: `cond` = 490.8, essentially that of the continuous system, and $\|J_{\text{sw}}\dot{q}^+\| = 4.8 \times 10^{-16}$.

The impact is violently dissipative. On the reference gait the kinetic energy falls $70.06 \rightarrow 20.27$ J — **71.1 % destroyed in an instant** — with an impulse of $[-11.85, 4.86]^\top$, norm 12.81 Ns (the Table 3.1 impulse quantity). This is the physical content of “plastic”: the ground absorbs whatever is required to stop the foot dead. It is also why the impact dominates the hybrid stability budget, and why ε in the stage-5 law is squeezed — the controller has exactly one step to undo what Δ does in zero time.

Order is not negotiable. Δ is impact, then relabel, then re-plant. Swapping the first two does not merely perturb the result:

wrong-order impulse	$[0.000, 0.000]$ versus correct $[-11.851, 4.858]$
velocity error	$\ \dot{q}_{\text{correct}}^+ - \dot{q}_{\text{wrong}}^+\ = 13.85$ (correct norm 5.49)
true swing foot	left sliding at 0.663 instead of 4.8×10^{-16}

The impulse is exactly zero — the impact becomes a no-op. The reason is worth understanding rather than memorizing: `ch3_relabel` swaps the leg coordinate slots, so after relabelling, J_{sw} evaluated on the new coordinates describes the *old stance* foot. That foot was pinned and already satisfies $J\dot{q} = 0$; the constraint the solver is asked to enforce is already met, the multiplier is zero, and nothing happens. The code runs and returns plausible numbers.

Re-plant, and the coordinate left alone. Since y is down-positive, $\text{foot}_z = -y - (\text{joint terms})$, so adding foot_z to y drives the new stance foot to exactly $z = 0$. On a collocation solution the correction is negligible (1.7×10^{-12} m — node N is constrained to the guard), but in forward simulation the event tolerance leaves a real residual, and without the re-plant the contact point

ratchets downward across steps. p_x is deliberately *not* reset (measured $\Delta p_x = 0$ through Δ): it is the translation gauge and must accumulate, which is why periodicity is imposed on 13 components and not 14.

7 Model verification

The generated dynamics were checked against physical invariants, finite differences, and independent second computations — no check reuses the pipeline’s own claims. Test set: three states from the reference gait plus four random physically plausible poses.

check	value	tolerance
M symmetric	0	10^{-10}
M positive definite (min. eigenvalue)	$4.21 \times 10^{-2} > 0$	—
$M_{11} = M_{22} = \text{total mass (30 kg)}$	0	10^{-10}
$DM = \dot{M}$ (finite difference)	3.05×10^{-9}	10^{-4}
passivity $\dot{q}^\top (\dot{M} - 2C)\dot{q} = 0$	1.05×10^{-14}	10^{-9}
$G = \partial P / \partial q$ (finite difference)	3.00×10^{-8}	10^{-5}
$J_{\text{st}}, J_{\text{sw}}$ versus finite differences	2.3×10^{-10}	10^{-6}
$\dot{J}\dot{q}$ (stance, swing) versus finite differences	3.2×10^{-9}	10^{-5}
$P_{\text{st}}, P_{\text{sw}}$ versus the T_2, T_4 transforms	1.1×10^{-16}	10^{-12}
HH6: θ equals the geometric leg angle (mod 2π)	5.33×10^{-15}	10^{-10}
$f_{1:7} = \dot{q}$ and $g_{1:7,:} = 0$	0	10^{-14}
$f + gu$ versus	4.55×10^{-13}	10^{-8}
rabbit_constrained_dynamics		
λ affine in u	1.14×10^{-13}	10^{-8}
$J\ddot{q} + \dot{J}\dot{q} = 0$	2.19×10^{-13}	10^{-8}
energy conserved at $u = 0$ (relative drift)	1.11×10^{-14}	10^{-8}
impact: $J_{\text{sw}}\dot{q}^+ = 0$	8.33×10^{-16}	10^{-10}
impact dissipates kinetic energy	$\Delta KE \leq 0$	—

Table 3: Model verification. The passivity identity is the strongest single check on V : it ties the Coriolis vector to the derivative of the mass matrix, two separately generated files. Energy conservation is the strongest end-to-end check: integrating the constrained dynamics for 0.25 s at $u = 0$ exercises M , V , G , J_{st} , $\dot{J}\dot{q}$ and the KKT solve simultaneously ($194.150857 \rightarrow 194.150857$ J).

The HH6 identity, stated precisely. The geometric identity is

$$\text{atan2}(-v_x, -v_z) = q_t + q_1 + \frac{1}{2}q_2 \pmod{2\pi}, \quad v = p_{\text{st}} - p_{\text{hip}}, \quad (16)$$

verified to 5.3×10^{-15} over 5000 random poses with either knee anywhere in $(-\pi, \pi)$. What is exact for all poses is the *direction*; the hip-to-foot *distance* is $\cos(q_2/2)$ (verified to 4.4×10^{-16}), so the leg shortens as the knee bends and equals the straight-leg length 1 only at $q_2 = 0$. Writing $L = \frac{1}{2}$ for the link length, $\sin a_1 + \sin(a_1 + q_2) = 2 \sin(a_1 + \frac{q_2}{2}) \cos \frac{q_2}{2}$ and likewise for the cosine, so $\cos(q_2/2)$ factors out as magnitude and the angle $a_1 + q_2/2$ is what remains.

7.1 Smoothness of ch3_yd

Because $d^2 y_a / ds^2$ enters $L_f^2 y$ directly, the output profile must be C^2 and the derivatives it returns must genuinely be its derivatives. Both were tested. The default Bézier basis passes unchanged;

the optional B-spline basis had two defects, **both since fixed** — the before/after contrast is drawn in Fig. 1(c,d) and described below, because it is the reason the code is written the way it is.

Bézier: smooth, and the derivatives are correct. Against central finite differences on $s \in [-0.4, 1.4]$ with $h = 10^{-5}$, $\max |dy_d - \frac{d}{ds}y_d| = 3.74 \times 10^{-9}$ and $\max |d^2y_d - \frac{d}{ds}dy_d| = 7.22 \times 10^{-9}$. (Comparing d^2y_d against the *second*-difference of y_d gives 1.05×10^{-4} , which is that formula's own roundoff, $\approx \epsilon/h^2$, not an error in `ch3_yd`.) Scanning 200 000 samples, every sample-to-sample step in y_d , dy_d and d^2y_d sits just under the bound implied by the next derivative — 1.624×10^{-4} against a Lipschitz bound of 1.640×10^{-4} , and so on — so there are no jumps. This is expected: a degree-5 polynomial is C^∞ . It is now checked rather than assumed.

B-spline: the same curve, two defects since corrected. At matching degree (`bsp_deg` = 5 with 6 control points) the B-spline reproduces the Bézier curve to 6.66×10^{-16} in value and 1.78×10^{-15} in slope, which confirms the degree-5 `BSpline_derivative` fix. Two further problems were found and repaired:

- **The clamp made dy_d inconsistent with y_d .** A clamped knot vector defines the curve only on $[0, 1]$, so unlike the Bézier branch this one must clamp s . It did — but then evaluated the slope at the clamped s as well, so outside the interval y_d froze constant while the reported dy_d froze at the *boundary slope* (0.561 below $s = 0$, 0.883 above $s = 1$). A constant function has zero derivative, so the returned dy_d was not the derivative of the returned y_d , and `ch3_outputs` then built $J_y = H - dy_d \partial s / \partial q$ from it and added a term that should not have been there. This also defeated the phase margin of Section 5: `ch3_phase` deliberately allows $s \in [-0.5, 1.5]$ because a polynomial is well behaved just outside $[0, 1]$, but the B-spline branch reinstated a hard clamp at exactly $[0, 1]$ — a kink at $s = 0$, the start of every step. *Fix:* dy_d and d^2y_d are now zero whenever the clamp engages, matching what `ch3_phase` does with $\partial s / \partial q$.
- **The second derivative degraded at touchdown.** It is obtained by differencing the analytic first derivative with $h = 10^{-5}$, and the stencil was *clipped* to $[0, 1]$ — which silently turns the central difference into a half-width one-sided formula, dropping it from $O(h^2)$ to $O(h)$. The error was flat at $\approx 4 \times 10^{-9}$ across the interior and jumped to 1.76×10^{-4} once $1 - s < h$. The affected neighbourhood is narrow, but $s = 1$ is exactly where node N sits and where $|d^2y_d/ds^2|$ is largest on this gait (5.85 versus ≈ 1 mid-step), so it fed $L_f^2 y$ at the worst moment. *Fix:* the stencil is now kept entirely inside $[0, 1]$, using a second-order one-sided formula within h of an end. The error at $s = 1$ falls from 1.76×10^{-4} to 3.77×10^{-9} , matching interior accuracy.

Neither defect was reachable in the default configuration (`p.basis = 'bezier'`), where no clamp exists and both derivatives are closed-form; they affected only the B-spline cross-check path. All five `ch3_test_*` suites pass after the change.

Two documentation defects found, neither affecting behaviour.

- `p.mass` in `Chapter3/ch3_params.m` read 32 against an actual total mass of 30 kg (10 torso + 4×5 links), confirmed both by summing `Mass_Properties` and by reading M_{11} , which for a translational floating-base coordinate *is* the total mass. The field is assigned and never read anywhere in the codebase. *Corrected to 30.*
- The `ch3_io_lin` knee-lock attribution discussed in Section 6. *Corrected* — the docstring now states the profile-slope mechanism the measurements support, drops the knee-lock claim they contradict, and records that the `pinv` path has never been observed to fire.

8 Trajectory optimization (the nonlinear program)

Decision vector.

$$z = \begin{bmatrix} \text{vec}(\text{coeffs}) \\ x_{\text{start}} \end{bmatrix} \in \mathbb{R}^{n_u n_c + 14} = \mathbb{R}^{38}, \quad (17)$$

with $n_u = 4$ actuated outputs and $n_c = 6$ control points each (`unpack_z.m`).

Cost (`hzd_cost.m`): squared torque per unit step length (Westervelt eq. 6.43),

$$J(z) = \frac{1}{L_{\text{step}}} \int_0^{T_{\text{step}}} \|u(t)\|_2^2 dt, \quad (18)$$

where L_{step} is the stance-to-swing foot distance at impact. Normalizing by L_{step} removes the degenerate incentive to “step in place”.

Constraints (`hzd_constraints.m`). Equalities ($\text{ceq} \in \mathbb{R}^{18}$):

- one-step periodicity, $x_{\text{next}}(2:14) = x_{\text{start}}(2:14)$, with $x_{\text{next}} = \Delta_{\text{reset}}(\Delta_{\text{impact}}(x_{\text{end}}))$ (13);
- stance foot on ground at $t = 0$ (1);
- stance foot zero velocity, $J_{\text{st}} \dot{q}_{\text{start}} = 0$ (2);
- touchdown phase, $\theta(q_{\text{end}}) = \theta^+$ (1);
- **NEC1** average walking rate, $L_{\text{step}}/T_{\text{step}} = v_{\text{des}}$ (1).

Inequalities ($c \in \mathbb{R}^8$, $c \leq 0$): no ground penetration; mid-step swing-foot clearance (NIC3); step-length floor; orbital stability (Section 10); and the four *physical realizability* constraints of Section 11 (torque, friction, ground reaction, impact impulse). The stability and physical constraints are each individually *gated* (default off) — see those sections. Disabled inequalities are held at a trivially satisfied value so c keeps constant length.

Solver (`hzd_problem_setup.m`). `fmincon` SQP with *central* finite differences (the event-triggered `ode45` makes forward differences noisy) and `ScaleProblem` *off* (so reported feasibility is directly comparable to the constraint tolerance). Cold starts from x_0 are unreliable; `build_gait_library.m` warm-starts each speed from the previously converged gait and marches v_{des} in small steps.

Diagnostics. `inspect_solution.m` prints and returns a full breakdown of any z : the θ sweep, step length/duration, walking speed, cost, every constraint residual, the worst violation, the within-step stance-foot drift, and the orbital-stability radius ρ (below).

9 Direct collocation

Everything above transcribes the gait problem by *single shooting*: the decision variables are the Bézier coefficients and one initial state, and the dynamics are satisfied implicitly because `ode45` integrates them. `Trajectory_Optimization/Collocation/` implements the alternative transcription — *Hermite-Simpson direct collocation* — which matches the thesis method. There is no `ode45` in the optimization loop at all.

What changes. The trajectory itself becomes a decision variable. For N nodes the vector z concatenates

block	size	meaning
X	$2n_q \times N$	node states $[q; \dot{q}]$
U	$n_u \times N$	node joint torques
Λ	$2 \times N$	node stance–foot contact forces $[F_x; F_z]$
coeffs	$n_u \times n_c$	Bézier virtual–constraint parameters
T	1	step duration

(`col_pack.m` / `col_unpack.m`). With the defaults $n_q = 7$, $n_u = 4$, $n_c = 6$, $N = 20$ that is $280 + 80 + 40 + 24 + 1 = 425$ variables, versus the 38 of the shooting formulation. The dynamics are then imposed as *equality constraints* — the Hermite–Simpson defects between consecutive nodes — rather than being integrated. The virtual constraints $y = q_{4:7} - y_d(s, \text{coeffs}) = 0$ are enforced at *every* node, so the solution lives on the hybrid zero–dynamics manifold by construction and **coeffs** still parametrizes it.

The trade. Collocation buys a much better conditioned problem: no event–triggered integrator inside the finite differences, so the Jacobian is sparse, exact in structure, and free of the integration noise that forced central differences in the shooting setup. It pays for this with an order of magnitude more variables and with a solution that is only *as feasible as its defect tolerance* — which motivates the cross–check below.

Warm starting (`rabbit_hzd_collocation.m`). The driver accepts either kind of saved gait and does the right thing with each:

- a *collocation* result (`col_result_*.mat`, identified by carrying an `N` field) has its z reused directly — N must match;
- a *shooting*/*PD* result (`hzd_result_*.mat`) is simulated for one step and resampled onto the N nodes by `col_seed_from_pd.m`.

This is what makes the “one constraint at a time” workflow practical: enable one physical constraint, re–solve warm–started from the last converged collocation gait, repeat. `col_verify_seed.m` reports the norm of every equality block on a seed *without solving*, which separates a genuine transcription bug from a merely hard NLP.

Cross–checking the result (`col_crosscheck.m`). A collocation solution is an **open–loop** trajectory. It satisfies the defect constraints, but nothing in the transcription guarantees that the shooting simulator, driven by an actual feedback law, reproduces it. The cross–check extracts **coeffs** and the node–1 state, simulates one step under `p.controller` (`'pd'` or `'clfqp'`), and overlays the two (Figure 3).

Stability of a collocation gait (`col_poincare.m`). Same Poincaré machinery as Section 10, fed from a `col_result_*.mat`. Use `'pd'`: the CLF–QP law solves a QP per ODE step, and ρ costs ≈ 26 step simulations.

10 Orbital stability: periodic is not the same as stable

This section is deliberately expanded for teaching, because it is the single most instructive point in the pipeline.

The trap. The periodicity equality makes x_{start} a *fixed point* of the step-to-step (Poincaré) map

$$x_{\text{next}} = P(x_{\text{start}}) := \Delta_{\text{reset}}(\Delta_{\text{impact}}(\varphi_{\text{step}}(x_{\text{start}}))), \quad (19)$$

where φ_{step} is one closed-loop swing phase. But a *fixed point can be unstable*. Satisfying periodicity to machine precision guarantees a limit cycle *exists*; it says nothing about whether the robot stays on it.

The diagnosis. Linearize (19) about the fixed point, $A = \partial P / \partial x$, and look at its spectral radius

$$\rho = \max_i |\lambda_i(A)|. \quad (20)$$

The gait is orbitally (asymptotically) stable iff $\rho < 1$. This is Westervelt’s stability requirement (informally, NEC5). For the reference seed gait obtained by enforcing periodicity *alone*, the numbers are stark:

max periodicity	ρ	open-loop $ \dot{q} $ over 6 steps
0.007 (excellent)	≈ 3 (unstable)	$6 \rightarrow 6 \rightarrow 19 \rightarrow 891 \rightarrow 949 \rightarrow 440$

The orbit is a near-perfect fixed point that the robot falls off after two steps. Every downstream symptom we observed — stance-foot drift growing across a step, Baumgarte constraint stabilization diverging, the Poincaré settling iteration blowing up, fragile speed continuation — is the same instability seen from a different angle: an orbit with $\rho > 1$ amplifies *every* perturbation, numerical or physical.

The remedy. Add $\rho < 1$ as an optimization constraint. `poincare_stability.m` builds A by central finite differences on the 13 periodic states (index 2:14; p_x is the translation gauge and is excluded),

$$A_{:,j} = \frac{P(x_{\text{start}} + h e_j) - P(x_{\text{start}} - h e_j)}{2h}, \quad j = 1, \dots, 13, \quad (21)$$

and returns $\rho = \max_i |\lambda_i(A)|$. `hzd_constraints.m` then adds the inequality $\rho - \rho_{\text{max}} \leq 0$ (with $\rho_{\text{max}} = 0.95$).

Cost and workflow. Each ρ evaluation costs $\approx 2 \times 13 = 26$ step simulations, and `fmincon` finite-differences the constraint over 38 variables — hundreds to thousands of simulations per iteration. The constraint is therefore **gated** behind `p.enforce_stability` (default `false`). The recommended workflow is two-phase:

1. solve a *periodic* gait with stability off (fast);
2. turn `p.enforce_stability = true` and re-solve *warm-started* from that gait to drive ρ below 1.

`inspect_solution.m` reports ρ on every call regardless of the gate, so stability is always visible even when it is not being enforced.

Analogy. Just as the *step-length floor* was the missing constraint that stopped the optimizer exploiting a degenerate “step in place”, the *stability bound* is the missing constraint that stops it returning a mathematically periodic but physically un-walkable orbit. In both cases the optimizer was doing exactly what it was told; the specification was incomplete.

11 Physical realizability (thesis Table 3.1)

Two families of constraints. It helps to see the whole set as doing two different jobs:

- **Mathematical validity** — is this a *real, attracting* gait? Periodicity (a fixed point), $\theta_{\text{end}} = \theta^+$ (the phase closes), NEC1 (walking rate), and orbital stability ($\rho < 1$, Section 10). These say nothing about physics: a gait can satisfy all of them and be impossible to build.
- **Physical realizability** — can the hardware and the ground actually produce it? The Table 3.1 constraints below.

Both are required, and “periodic” (cheap to satisfy) is a tiny subset of “walkable” (both families). All of the physical quantities are recovered from the *same* simulation the optimizer already runs: the contact force $\lambda = [F_x; F_z]$ is an output of `rabbit_constrained_dynamics`, the joint torque u of `hzd_control_and_dynamics`, and the impact impulse is the multiplier inside `rabbit_impact_map`. `gait_forces.m` replays the trajectory and returns the peaks; `hzd_constraints.m` turns them into inequalities.

constraint	form (value)	what failure it prevents
Vertical GRF	$\min F_z \geq F_{z,\min} (\geq 20 \text{ N})$	ground can only <i>push</i> (unilateral contact); $F_z \leq 0$ means the foot lifts / the model invents a pulling force
Friction cone	$\max F_x/F_z \leq \mu (0.4)$	stance foot <i>slipping</i> (well posed only where $F_z > 0$)
Joint torque	$\max u \leq u_{\max} (120 \text{ Nm})$	commanding torque the actuator cannot deliver
Impact impulse	$ F_e \leq F_{e,\max} (30 \text{ Ns})$	a violent foot-strike (hardware damage, hard landing)
Swing clearance	$h_f \geq h_{\min} (0.1 \text{ m, NIC3})$	scuffing / premature re-contact (a “step in place”); ensures S is hit once, at touchdown
Dynamic constr.	EoM	(free here — we integrate the ODE, so it holds by construction)

Values do not transfer from the reference. The thesis numbers ($|u| \leq 5 \text{ Nm}$, $F_z \geq 200 \text{ N}$, $F_e \leq 15 \text{ Ns}$) are for ATRIAS: **63 kg with 50:1 harmonic drives**, so its 5 Nm *motor* torque is 250 Nm at the joint. RABBIT here is $\sim 30 \text{ kg}$ with **no gear ratio** (u is the joint torque directly), so the limits must be re-derived — measure first (`inspect_solution` prints all four), then set bounds.

Order matters — a worked diagnosis. Measured on the reference gait, the quantities were:

quantity	measured	verdict
peak $ u $	257 Nm	high ($> u_{\max}$)
mean F_z	+309 N	$\approx$ body weight, correct sign
min F_z	−193 N	negative: the foot lifts off
max $ F_x/F_z $	255	artifact of $F_z \rightarrow 0$ (not real slip)
$ F_e $	9.8 Ns	fine

The instructive point: the enormous friction ratio is *not* a friction problem. $|F_x/F_z|$ diverges wherever F_z crosses zero, and here F_z goes negative — physically impossible, since the ground can only push. So the *ground-reaction* constraint ($F_z > 0$, foot in compression) is the root; the friction ratio only becomes meaningful once it holds (and `gait_forces` therefore samples $|F_x/F_z|$ only where $F_z > 1$ N). Enabling friction first (before fixing F_z) made `fmincon` chase a divide-by-zero and diverge; enabling *GRF first* is correct. Each constraint has its own toggle (`p.enforce_grf`, `enforce_friction`, `enforce_torque`, `enforce_impulse`) precisely so they can be introduced one at a time, warm-started, in the physically sensible order.

The constraints are coupled, not four independent knobs. Two couplings are worth internalizing. First, *GRF is upstream of friction*: the ratio $|F_x/F_z|$ diverges as $F_z \rightarrow 0$, so friction is only well posed once $F_z > 0$ is enforced — the ordering is dictated by the math, not taste. Second, *torque is upstream of almost everything*: a tighter torque budget caps joint accelerations, which caps how hard the robot throws its mass, which shrinks GRF spikes and friction demand. A violent gait therefore violates the whole table at once — which is exactly what the reference seed does (257 Nm, F_z changing sign, $\rho \approx 129$). These are four views of one question: *is this gait gentle enough to be real?*

Caveat. The reference gait violates almost every one of these by large margins *and* is strongly unstable ($\rho \gg 1$). Repairing it incrementally may not converge from such a pathological seed; a materially better initial gait, or the reduced 2-D zero-dynamics formulation, is the likely path to a gait that is simultaneously periodic, stable, and physically realizable.

Synthesis: constraints as specification completeness. Every constraint beyond the raw equations of motion was added because the optimizer found a loophole in an incomplete specification, and each loophole was a *different* flavour of “technically periodic”:

1. periodic but **degenerate** (step in place) $\rightarrow$ step-length floor;
2. periodic but **unstable** (falls after two steps) $\rightarrow$ stability, $\rho < 1$;
3. periodic but **unphysical** (foot lifts, slips, over-torque) $\rightarrow$ Table 3.1.

The optimizer was never wrong; it did exactly what it was told, and each surprising result exposed a missing line in the specification. Table 3.1 is best read not as “extra realism” bolted on, but as *the rest of the definition of walking* — the part that is obvious to a human and invisible to `fmincon` until it is written down.

12 Software architecture and data flow

13 File reference

Dynamics/ (mostly Symbolic-Toolbox generated)

file	role
<code>rabbit_energy_model_generators.m</code>	Generalized Lagrange-Lagrangian derivation; defines masses/inertias; produces $T_t, T_1\text{--}T_4, P_{st}, P_{sw}, M, DM, V, G$.
<code>Jacobians.m</code>	Generator: produces $J_{st}, J_{sw}, J_{dotdq_{st}}, J_{dotdq_{sw}}$.

file	role
Tt.m, T1-T4.m	Homogeneous transforms of torso / link frames, $T=T_t(q)$.
P_st.m, P_sw.m	Stance / swing foot position $[x; z]$, world frame.
M.m	Mass matrix $M(q)$ (7×7).
V.m	Coriolis/centrifugal vector, $V([q; dq])$.
G.m	Gravity vector $G(q)$.
DM.m	$\dot{M}$, $DM([q; dq])$ (used to form V).
J_st.m, J_sw.m	Foot Jacobians (2×7).
Jdotdq_st.m, Jdotdq_sw.m	$\dot{J}\dot{q}$ terms (2×1).
input_matrix.m	Constant actuation map B .
rabbit_constrained_dynamics.m	Single-support forward dynamics (KKT solve), $[ddq, \lambda] = \dots(q, dq, u)$; optional Baumgarte via a 4th <code>foot_ref</code> arg.
hzd_control_and_dynamics.m	Control law + constrained dynamics for one state; returns $[ddq, \tau, \lambda]$ (lets torque/force be replayed post-sim).
hzd_closed_loop_ode.m	Closed-loop swing RHS; wraps the above and appends the torque-cost integrand.
rabbit_ode.m	Generic single-support RHS for an arbitrary controller.
rabbit_dynamics.m	Broken : calls nonexistent <code>D_matrix/C_vector/G_vector</code> .

Contact/ and Reset_Map/

file	role
rabbit_impact_event.m	ODE event: swing foot reaches ground.
impact_event_wrapper.m	Same event for the augmented HZD state ξ .
rabbit_impact_map.m	Momentum-conserving velocity reset, $[x_{plus}, impulse] = \dots(x_{minus})$ (impulse is the optional 2nd output).
relabel_state.m	Swap stance/swing joints and velocities.
rabbit_reset_map.m	Relabel + re-plant stance foot on ground.

Controller/

file	role
theta_of_q.m	Phase variable $\theta = q_t + q_1 + \frac{1}{2}q_2$.
dtheta_dq_of.m	Constant gradient c .
rabbit_controller.m	Broken : calls nonexistent <code>desired_gait</code> .

Trajectory_Optimization/ (main pipeline)

file	role
<code>rabbit_hzd_trajectory_optimization.m</code>	Entry point: single-gait solve, save, animate.
<code>build_gait_library.m</code>	Speed continuation; builds a <code>gait_library</code> struct.
<code>hzd_problem_setup.m</code>	Single source of p , bounds, <code>fmincon</code> options.
<code>hzd_cost.m</code>	Objective (torque ² per step length).
<code>hzd_constraints.m</code>	Equality + inequality constraints.
<code>unpack_z.m</code>	Split $z = [\text{coeffs}; x_{\text{start}}]$.
<code>make_coeffs.m</code>	Initial B-spline control points.
<code>bspline_eval.m</code>	Evaluate a spline output and its s -derivative.
<code>BSpline.m</code> , <code>BSpline_derivative.m</code>	Cox-de Boor basis and derivative.
<code>inspect_solution.m</code>	Full diagnostic breakdown of a solution (incl. ρ).
<code>poincare_stability.m</code>	Step-map Jacobian; returns spectral radius ρ (NEC5).
<code>gait_forces.m</code>	Replays a gait; returns peak torque, min/mean vertical GRF, friction ratio, impact impulse (Table 3.1).
<code>settle_initial_state.m</code>	Poincaré iteration (disabled by default).

Trajectory_Optimization/Collocation/ (Section 9)

file	role
<code>rabbit_hzd_collocation.m</code>	Entry point: Hermite-Simpson collocation solve, warm-started from a PD or collocation gait.
<code>col_problem_setup.m</code>	Extends p with the node count N and the collocation-vector bounds.
<code>col_pack.m</code> , <code>col_unpack.m</code>	Assemble / split $z = [X; U; \Lambda; \text{coeffs}; T]$.
<code>col_dynamics.m</code>	Constrained EoM evaluated at a node (state, torque, contact force).
<code>col_cost.m</code>	Objective on the node torques.
<code>col_constraints.m</code>	Hermite-Simpson defects, virtual constraints at every node, foot pinning, periodicity.
<code>col_seed_from_pd.m</code>	Simulate a PD gait and resample it onto N nodes to build z_0 .
<code>col_verify_seed.m</code>	Report every equality-block norm on a seed <i>without</i> solving (transcription sanity check).
<code>col_crosscheck.m</code>	Open-loop collocation trajectory vs. a closed-loop sim (Figure 3).
<code>col_poincare.m</code>	Spectral radius ρ of a collocation gait under 'pd' or 'clfqp'.

Simulation/, Utilities/, Visualization/

file	role
<code>simulate_hzd_gait.m</code>	One HZD step, summary outputs (status, penetration, clearance, T_{step}).
<code>simulate_hzd_gait_full.m</code>	One HZD step, full trajectory (for animation/drift).
<code>simulate_one_step.m</code>	Generic one step for an arbitrary controller.
<code>simulate_n_steps.m</code>	Chain generic steps with impact+reset.
<code>get_body_points.m</code>	World positions of key body points.
<code>check_ground_validity.m</code>	Ground/penetration/slip validation of a state.
<code>animate_rabbit.m</code>	Stick-figure animation + GIF export.
<code>animate_hzd_result.m</code>	Stitch n HZD steps and animate.

config/ and Test/

file	role
<code>make_initial_state.m</code>	Deterministic ground-valid start state (1 output).
<code>make_random_initial_state.m</code>	Rejection-sampled random start (unseeded).
<code>config.m</code>	Singleton: stepping-stone terrain + spline params.
<code>init_bspline_params.m</code>	Legacy B-spline controller config.
<code>test_simulate_steps.m</code>	Smoke test of the generic simulation path.

14 Reproducibility

- **Generated dynamics.** $T_t, T_1-T_4, P_{\text{st}}, P_{\text{sw}}, M, DM, V, G$ come from `rabbit_energy_model_generalized`, $J_{\text{st}}, J_{\text{sw}}, J_{\text{dotdq_st}}, J_{\text{dotdq_sw}}$ from `Jacobians.m`. *If any mass, length, inertia, or kinematic relation changes, these generator scripts must be re-run to regenerate the committed files. Do not hand-edit the generated files.*
- **Saved results.** `Results/hzd_result_<timestamp>.mat` each store $z_{\text{opt}}, p, f_{\text{val}}, \text{exitflag}, \text{report}$. The 2026-07-20_12-06-36 result is the reference periodic gait (≈ 0.355 m/s) used as the warm-start seed. `build_gait_library.m` writes `gait_library_<timestamp>.mat`. Collocation solves (Section 9) write `Results/col_result_<timestamp>.mat`, which additionally carry N — that field is how `rabbit_hzd_collocation.m` tells a collocation warm start from a PD one.
- **Default result selection is by filename, not mtime.** Every entry point that resolves a default `Results/*.mat` — `rabbit_hzd_collocation`, `col_crosscheck`, `col_poincare`, `col_verify_seed`, `Test/test_clf_qp` — picks `max` of `sort([d.name])`, *not* `max([d.datenum])`. The timestamp embedded in the filename is zero-padded, so a lexicographic sort is chronological, whereas filesystem mtime is not checkout-invariant: a fresh `git clone` or `git worktree add` stamps every checked-out file with the same time, and an mtime `max` then breaks the tie arbitrarily — in practice returning the *oldest* gait. Selecting by mtime therefore silently seeds a solve from a stale result and makes the outcome depend on which checkout it runs in.
- **Figures.** The result figures in this document (Figures 2, 3, 5, 4) are not hand-drawn: they are produced by `Visualization/make_doc_figures.m` from the newest `Results/col_result_*.mat`

— here `col_result_2026-07-23_00-45-04.mat` — into `docs/figures/`. After the reference gait changes, re-run that script and recompile (pdf~~l~~atex twice, for the cross-references) so the figures and the prose cannot drift apart.

- **Nondeterminism.** `make_random_initial_state.m` uses `randn` with no seed. The HZD pipeline instead uses a hard-coded x_0 , so its results are reproducible. Add `rng(seed)` before random sampling if determinism is needed there.
- **Orphaned cache.** `Trajectory_Optimization/Trajectory.mat` is committed but loaded by no code path; treat as unverified.
- **Simscape.** `Dynamics/Test_dynamics_Simscape.slx` is a standalone dynamics cross-check, not part of the runtime pipeline.

15 Known issues and broken legacy entry points

The following predate the HZD pipeline and **do not run as-is** — they call functions that no longer exist. They are retained for reference only.

- `main_demo.m`: calls `make_initial_state` expecting 3 outputs (it returns 1), `simulate_n_steps` with 4 arguments (it takes 3), and `animate_rabbit_stepping_stones` (does not exist).
- `Controller/rabbit_controller.m`: calls `desired_gait` / `desired_gait_velocity` (do not exist).
- `Dynamics/rabbit_dynamics.m`: calls `D_matrix` / `C_vector` / `G_vector` (do not exist; the real terms are M / V / G).

Open item: stance-foot constraint drift (a symptom). The stance foot is held only by the acceleration-level contact constraint, so `ode45` permits numerical drift of the foot within a step (~ 0.1 – 0.3 m; `inspect_solution.m` reports it). The natural fix is Baumgarte stabilization (`rabbit_constrained_dynamics.m` supports it via an optional `foot_ref`), but on the reference gait it *diverged* rather than helping. That is not a flaw in Baumgarte: as Section 10 shows, the seed orbit has $\rho \approx 3$, so it amplifies the very corrections Baumgarte injects. The drift is thus best read as *another symptom of orbital instability*; enforcing $\rho < 1$ addresses the root cause, after which constraint stabilization becomes well behaved.

Open item: enforcing stability is expensive. The $\rho < 1$ constraint is finite-difference heavy (Section 10). It is usable as a warm-started final phase but slow for a from-scratch solve; a reduced (2-dimensional hybrid zero dynamics) stability condition, or an analytic Poincaré Jacobian, would make it cheap. This is the natural next step for the pipeline, alongside the CLF-QP controller route (which stabilizes an open-loop-unstable orbit with feedback instead of redesigning it).

References

1. E. R. Westervelt, J. W. Grizzle, C. Chevallereau, J. H. Choi, B. Morris, *Feedback Control of Dynamic Bipedal Robot Locomotion*, CRC Press, 2007.

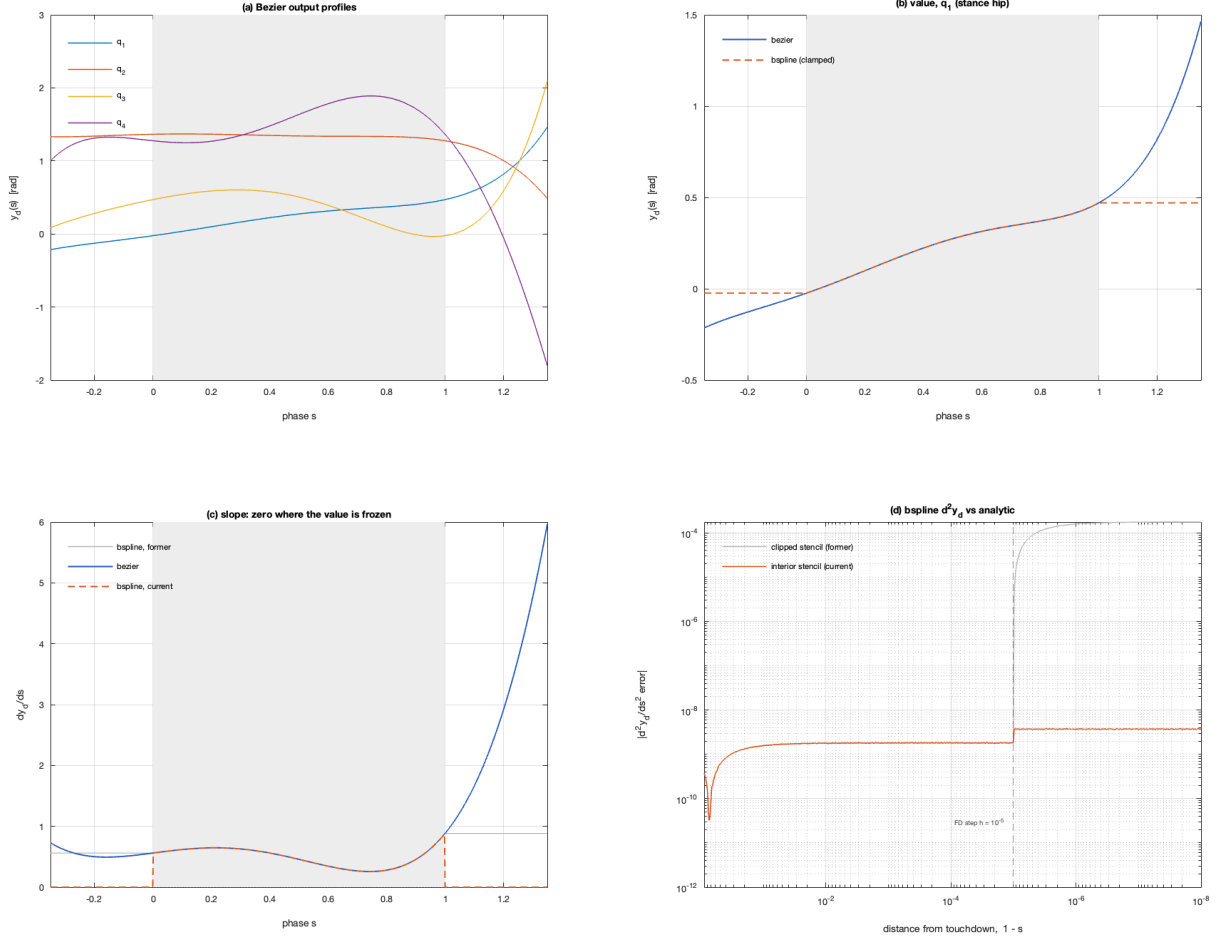


Figure 1: Smoothness of `ch3_yd`; shaded band is the nominal phase interval $[0, 1]$. (a) the four Bézier profiles, which extrapolate smoothly outside the band — this is what the phase margin relies on, and the steep divergence of q_4 past $s \approx 1.2$ is why that margin is 0.5 and not larger. (b) the B-spline value necessarily freezes outside $[0, 1]$, where the Bézier continues. (c) the reported slope: *former* behaviour froze at the boundary value, contradicting the constant it accompanied; *current* behaviour is zero wherever the value is frozen. (d) B-spline $d^2 y_d$ error against the analytic value, plotted against distance from touchdown (note the reversed logarithmic axis): the clipped stencil degenerated to one-sided once $1 - s$ fell below the step $h = 10^{-5}$ and lost five orders of accuracy; the interior stencil holds interior accuracy all the way to $s = 1$. Regenerate with `Visualization/make_yd_smoothness_figure.m`.

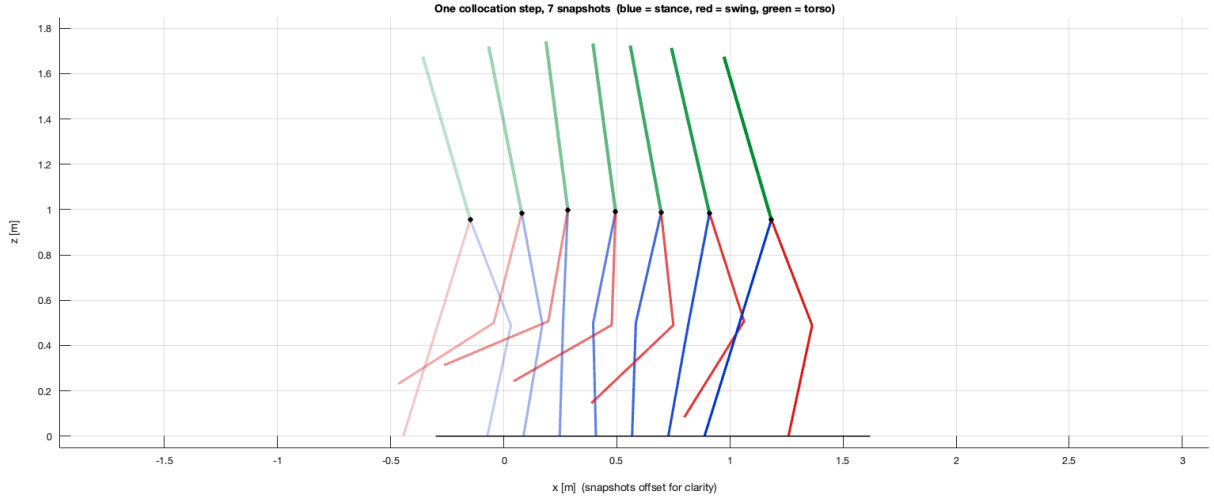


Figure 2: One converged collocation step (`col_result_2026-07-23_00-45-04.mat`, $N = 20$, $T = 1.046$ s), sampled at seven nodes and offset horizontally for legibility. Blue is the stance leg, red the swing leg, green the torso; opacity increases through the step. The stance foot stays planted while the swing leg advances and re-strikes ahead of it.

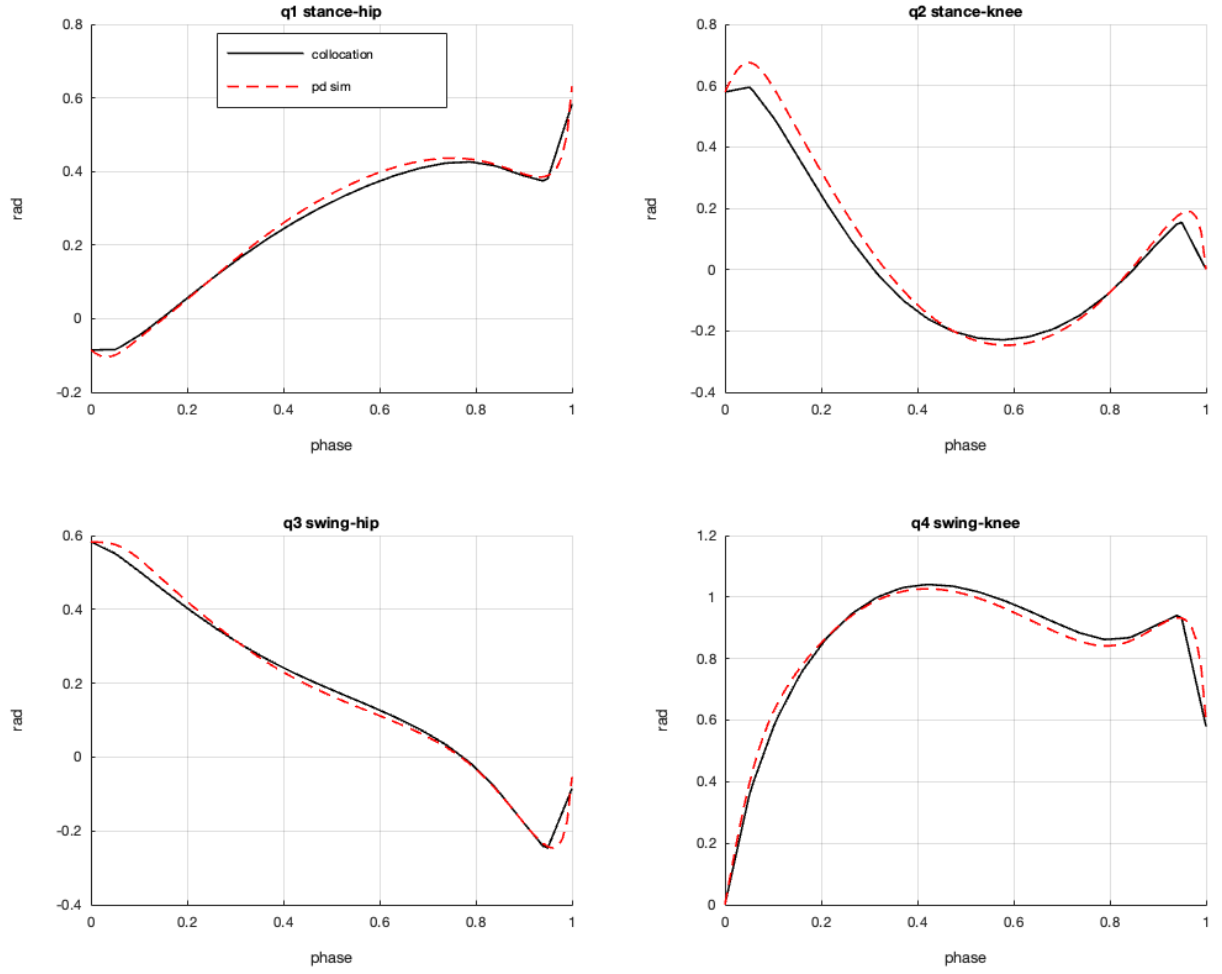


Figure 3: Collocation reference (black) versus the PD closed-loop simulation (red dashed) over one step, on a normalized phase axis. Tracking is close — 0.026 rad rms, 0.140 rad max, the largest errors at the stance knee just after impact — so the open-loop trajectory really is realizable by the feedback law. Note this says nothing about what happens on the *next* step; see Figure 4.

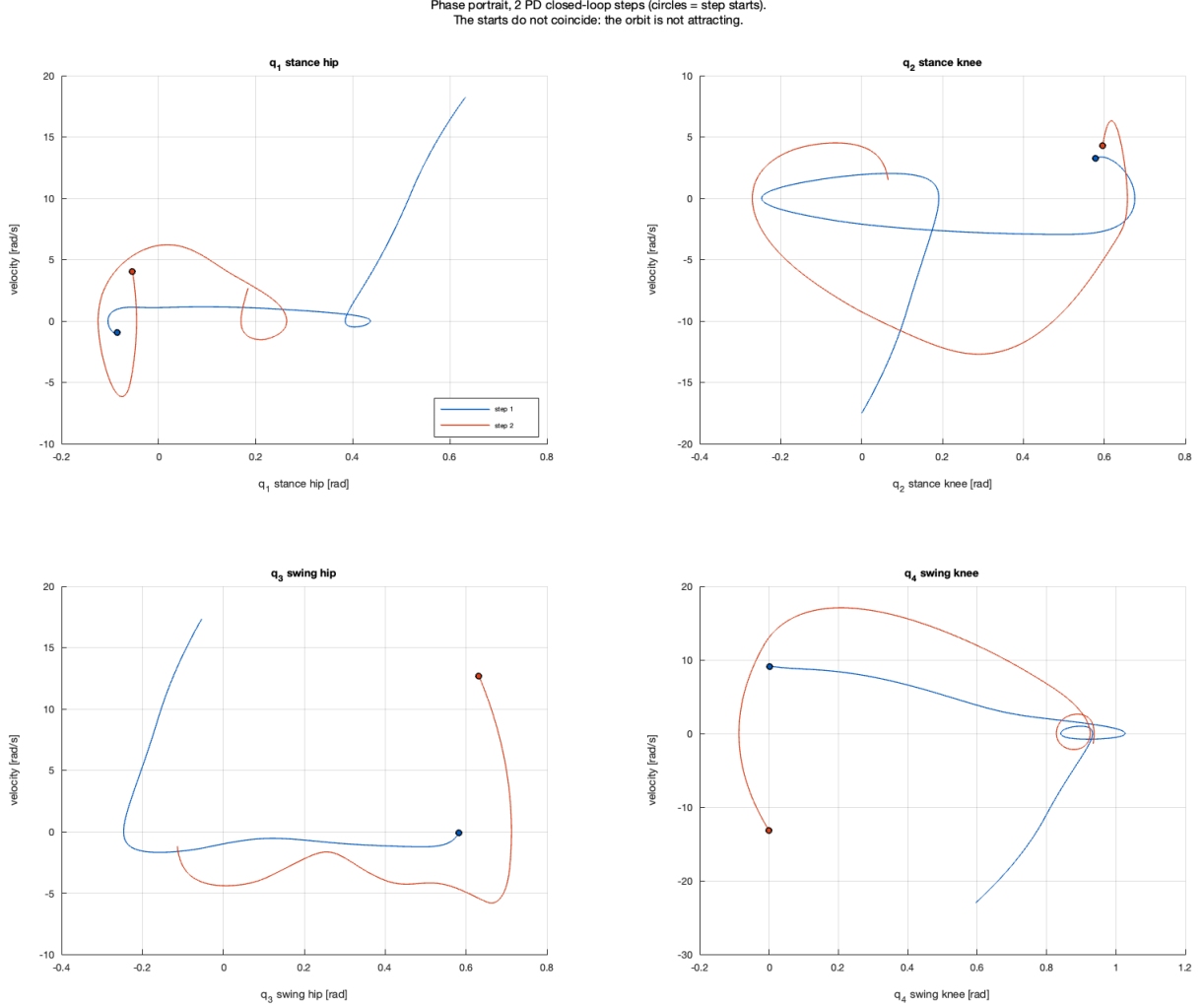


Figure 4: What $\rho > 1$ looks like. Phase portraits of the four actuated joints over consecutive PD closed-loop steps, started from the collocation gait's node-1 state. Circles mark where each step *begins*: on an orbitally stable limit cycle they would coincide, and instead step 2 starts visibly away from step 1 — most starkly at the swing hip, which restarts with ≈ 12.7 rad/s where step 1 began near zero. Only two steps are drawn because the third leaves the physical range entirely ($|q|$ reaching several hundred radians), the same blow-up tabulated above. Compare Figure 3: a *single* step tracks its reference well, which is exactly why one-step agreement is not evidence of a walkable gait.

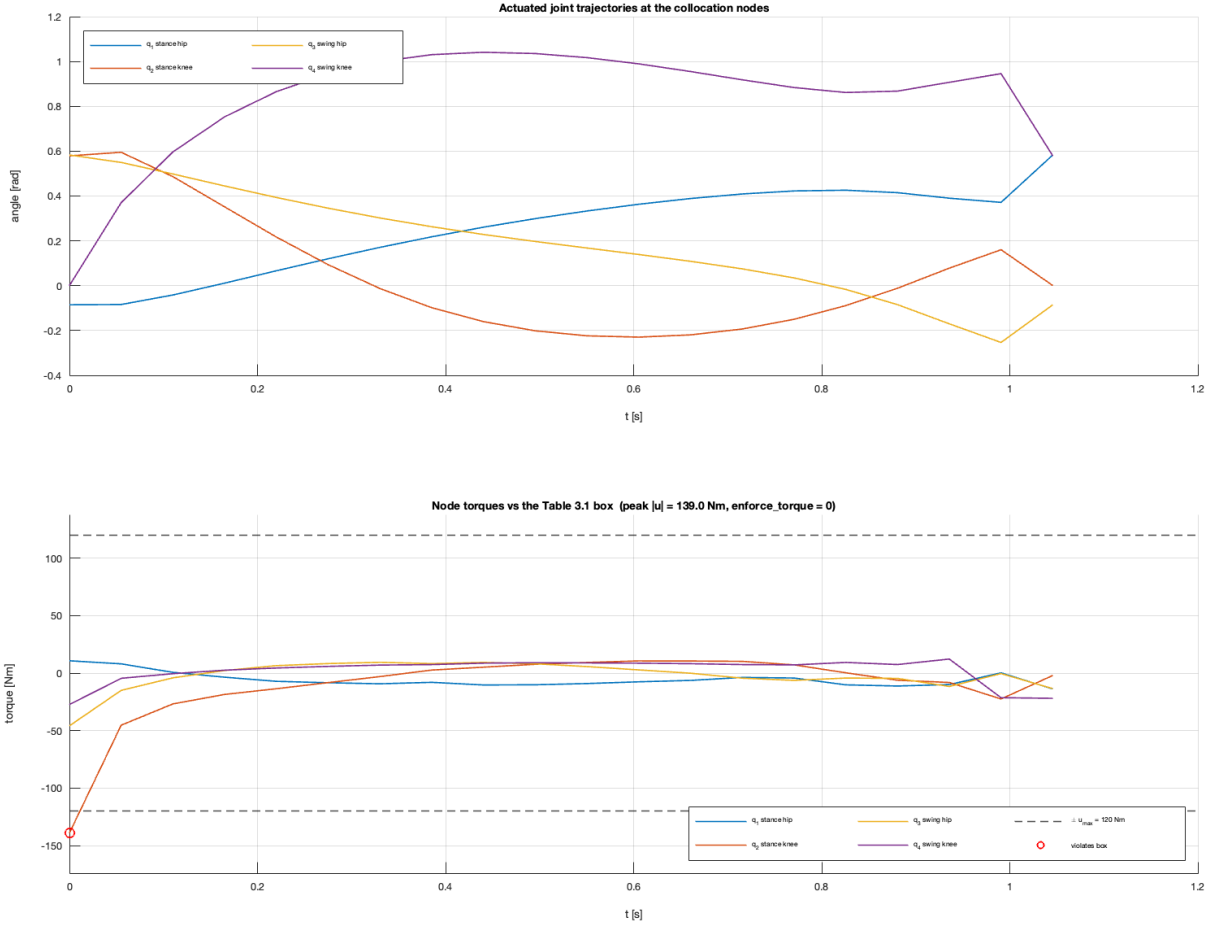


Figure 5: A gait caught *mid-way* through that one-at-a-time workflow. The saved collocation result has `enforce_grf` and `enforce_impulse` on but `enforce_torque` still **off**, and the lower panel shows the consequence: exactly one of the 80 node torques leaves the ± 120 Nm box (circled) — the stance knee at the first node, at -139.0 Nm, a 16% overshoot. With the toggle off this is not a solver failure but an *unconstrained* quantity, and it is precisely what enabling `enforce_torque` on the next warm-started solve is there to remove. The remaining 79 nodes sit well inside the box, which is why the violation is easy to miss without plotting it.

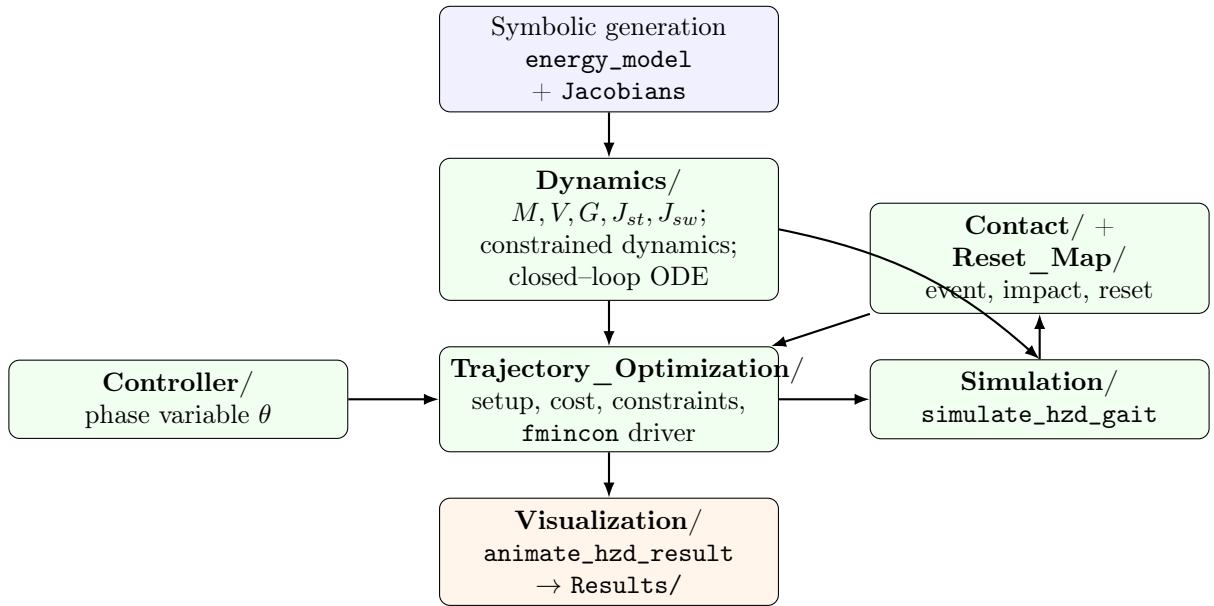


Figure 6: Pipeline: offline symbolic generation feeds the **Dynamics** layer; the optimizer repeatedly calls the single-step simulator (which integrates the closed-loop dynamics and detects impact) to evaluate the cost and constraints; the converged gait is animated and saved.